Two programs: Gemini and Copilot

Access: Gemini and Copilot in the Chrome browser from the school computer in the lab room

Prompt used:
Write a C program that picks a fixed secret number between 1 and 10. The user gets up to 3 tries to guess the number.

• Use a fixed secret number (for example, 7).
• Use printf to prompt the user and scanf to read their guess.
• Use a while or for loop to let the user guess up to 3 times.
• Use if-else statements to check if the guess is too high, too low, or correct.
• Stop the loop immediately if the user guesses the correct number.
• Print a final win or lose message.

Gemini#

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int max_tries = 3;
    int won = 0;

    for (int i = 1; i <= max_tries; i++) {
        printf("Attempt %d of %d - Enter your guess (1-10): ", i, max_tries);
        if (scanf("%d", &guess) != 1) {
            printf("Invalid input.\n");
            return 1;
        }

        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < secret) {
            printf("Too low.\n");
        } else {
```

```
            printf("Too high.\n");
        }
    }

    if (!won) {
        printf("You lose! The secret number was %d.\n", secret);
    }

    return 0;
}
```

Copilot#

```
#include <stdio.h>

int main() {
    int secret = 7;        // fixed secret number
    int guess;
    int tries = 0;

    printf("I'm thinking of a number between 1 and 10.\n");

    while (tries < 3) {
        printf("Enter your guess: ");
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            return 0;   // stop immediately
        } else if (guess < secret) {
            printf("Too low.\n");
        } else {
            printf("Too high.\n");
        }

        tries++;
    }

    printf("Sorry, you lose. The number was %d.\n", secret);
    return 0;
}
```

correctness

Both the Gemini and Microsoft Copilot programs are running correctly. Both use a fixed secret number of 7. But the problem is that the correct answer is always 7; it's not a random number that the user plays ten times and gets the same secret number, which is a little boring. We can try, each time running the program, to get a random, fixed secret number.And it didn't show how many guesses are allowed for the user
And neither of them had a reminder to tell the user that the range is between 1 and 10 if the user types in an integer greater than 10

Gemini:
Use printf() and scanf(), allow up to three guesses, use if-else statements to compare the guess with the secret number, and stop when the user guesses correctly. A slight difference is that Gemini uses for loops, and Copilot uses while loops

And both AIs can run the code but an advantage of Gemini is that it checks whether it read an integer or not using scanf()
if (scanf("%d", &guess) != 1), which allows the program to detect invalid input

Copilot,
It can run the program correctly, but not that perfectly
It uses the variable as the secret number prinf (),scanf(), a while loop
However, it didn't check the return value of scanf(), so if we type in the string, it will be invalid and break down

Gemini gets the point

For the execution time
Both programs allow 3 guesses as the maximum number of tries
Gemini uses a for loop; Copilot uses a while loop. Both time complexities are O(1),
But typically, for loops are faster than while loops.However, in this function we can ignore this.
Space complexity
Both programs use only a few of the integer variables
And the space complexity is O (1) for both

For maintainability
Both have almost the same time and space complexity
Gemini::
Gemini uses int max tries=3 in the loop, and outputting it makes the program easier to change and fix
If the number needs to be changed from 3 to 5, it can easily be changed and has clear readability
But the Gemini code doesn't have comments, which makes it harder to understand. We can add a prompt to let Gemini add comments on each line of the code
Copilot

while(tries<3)
The Copilot way of naming each variable makes us a little confused because this is <3 should be the max tries, but it names the variable as try instead of max_try, which reduces readability and makes it harder to add a single command to the prompt to fix. And the code itself has more problems than Gemini's one, which can't detect the non_integer input

For maintainability and correctness, Gemini is better; I will choose Gemini

How to improve:
correctness
Adding a range check to make sure the user's guess is between 1 and 10. Generate a random number between 1 and 10. And let the user know how many tries remain .And i probably won't count the trying times which the guess is out of the range of 1~10
Complexity time:
The Gemini program already has constant execution time because the user can make at most three guesses. Therefore, its time complexity remains O(1) I did not make unnecessary changes
Time complexity:
The program uses only a fixed number of integer variables and does not require arrays or other growing data structures. Therefore, its space complexity remains O(1), and it uses the minimum number of variables I can get
Maintainability
I will add the prompt of letting Gemini create the comment for the code which make the easier to read

```
/*
 * Program Name: EECS 348 Assignment 1
 * Description: A number guessing game where the user has up to
 *              three valid attempts to guess a randomly generated
 *              number between 1 and 10.
 * Inputs: The user's guesses entered through the keyboard.
 * Output: Messages indicating whether the guess is too high,
 *         too low, correct, invalid, and the number of tries remaining.
 * Collaborators: None
 * Other Sources:
 *   Google Gemini: https://gemini.google.com/
 *   Microsoft Copilot: https://copilot.microsoft.com/
 * Author: yunjing ma
 * Creation Date: September 8, 2026
 * Revision Date: September 8, 2026
 * Revisions: Added a random secret number, range checking,
 *            remaining-tries messages, and detailed comments.
 */
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    // Initialize the random number generator using the current time.
    srand(time(NULL));

    // Generate a random secret number between 1 and 10.
    int secret = rand() % 10 + 1;

    // Store the user's current guess.
    int guess;

    // Store the maximum number of valid guesses allowed.
    int max_tries = 3;

    // Store the number of valid guesses the user has made.
    int tries = 0;

    // Track whether the user has correctly guessed the secret number.
    int won = 0;

    // Continue asking for guesses until the user wins or uses all
    // three valid attempts.
    while (tries < max_tries) {

        // Ask the user to enter a number between 1 and 10.
        printf("Attempt %d of %d - Enter your guess (1-10): ",
               tries + 1, max_tries);

        // Read the user's input and check whether it is an integer.
        if (scanf("%d", &guess) != 1) {

            // Display an error message for non-integer input.
            printf("Invalid input. Please enter an integer between 1 and 10.\n");

            // Clear the invalid characters from the input buffer.
            int ch;
            while ((ch = getchar()) != '\n' && ch != EOF) {
                // Remove invalid characters from the input buffer.
            }
```

```
    // Do not increase tries because the input was not a valid guess.
    continue;
}

// Check whether the guess is outside the valid range.
if (guess < 1 || guess > 10) {

    // Tell the user that the guess must be between 1 and 10.
    printf("Invalid guess. Please enter a number between 1 and 10.\n");

    // Do not increase tries because the guess was out of range.
    continue;
}

// Increase the number of attempts only after receiving
// a valid guess between 1 and 10.
tries++;

// Check whether the user's guess is correct.
if (guess == secret) {

    // Tell the user that they guessed the number correctly.
    printf("Correct! You win!\n");

    // Record that the user won the game.
    won = 1;

    // Stop the loop immediately after a correct guess.
    break;
}

// Check whether the guess is lower than the secret number.
else if (guess < secret) {

    // Tell the user that the guess is too low.
    printf("Too low.\n");
}

// If the guess is not correct or too low, it must be too high.
else {

    // Tell the user that the guess is too high.
    printf("Too high.\n");
}
```

```
        // Display the number of valid attempts remaining.
        printf("You have %d %s remaining.\n",
               max_tries - tries,
               (max_tries - tries == 1) ? "try" : "tries");
    }

    // Check whether the user failed to guess the secret number.
    if (!won) {

        // Display the final losing message and reveal the secret number.
        printf("You lose! The secret number was %d.\n", secret);
    }

    // End the program successfully.
    return 0;
}
```

correctness

I improved the program's correctness in several ways. First, I changed the fixed secret number to a randomly generated number between 1 and 10, so the secret number can be different each time the program runs. Second, I added a range check to make sure the user's guess is between 1 and 10. Guesses outside this range are rejected and do not count as attempts. Finally, I added a message to tell the user how many valid tries remain after an incorrect guess. These changes make the program more flexible and provide clearer feedback to the user.

Complexity time:

The Gemini program already has constant execution time because the user can make at most three guesses. Therefore, its time complexity remains O(1) I did not make unnecessary changes

Time complexity:

The program uses only a fixed number of integer variables and does not require arrays or other growing data structures. Therefore, its space complexity remains O(1), and it uses the minimum number of variables I can get

Maintainability

I asked Gemini to generate detailed comments for the code. The comments explain the purpose of the variables and the program's major sections, making the code easier to read, understand, and maintain.